

Universidade da Beira Interior

Departamento de Informática



2025/2026: RAT Analyzer — Integrated Static and Dynamic Analysis Platform for Remote Access Trojan Detection

Elaborado por:

José Miguel Gonçalves de Amoreira — 51712

Orientador:

Professor João Bernardo Ferreira Sequeiros

24 de Junho de 2026

Abstract

This report presents RAT ANALYZER, an integrated static and dynamic analysis platform for detecting *Remote Access Trojans* (RATs) in Windows executables (and C# source with reduced scope), developed as the final-year project in Computer Science Engineering at Universidade da Beira Interior. The solution combines a Python backend (FastAPI) with a seven-phase pipeline (PE metadata, ILSpy, heuristics, deobfuscation, YARA, risk scoring, native/Ghidra when needed, and reporting), a React web interface (with optional Gemini assistant), a WPF desktop application, and an isolated Hyper-V sandbox with two independent orchestration paths (HTTP vm-agent and PowerShell Direct with SHA-256 integrity checks). Validation relies on 220 automated tests, proxy detection metrics on 59 labelled synthetic profiles stratified across five severity levels (precision/recall/F1 and level accuracy 1.00), a benign corpus of eight compiled PEs analysed statically (specificity 1.00, zero false positives), public MalwareBazaar IOC metadata (without host-side malware storage), a formal static benchmark (median 36.1 s, $n = 5$), synthetic scoring profiles aligned with `test_risk_scoring.py`, and benign execution via `benign-vm-test.exe` on Path B. The system was designed for explainability and local isolation rather than commercial cloud sandboxes, targeting academic and laboratory use.

Keywords: malware analysis; RAT; sandbox; Hyper-V; YARA; FastAPI; software engineering; cybersecurity.

Resumo

Este relatório apresenta o RAT ANALYZER, uma plataforma integrada de análise estática e dinâmica para detecção de cavalos de Troia de acesso remoto (RAT) em executáveis *Windows* (e código *C#* com âmbito reduzido), desenvolvida no âmbito do projeto final de Licenciatura em Engenharia Informática na Universidade da Beira Interior. A solução combina um servidor (*backend*) em *Python*, com *FastAPI*, e uma cadeia de processamento (*pipeline*) de sete fases (metadados de executáveis portáteis (*PE*), descompilação com *ILSpy*, heurísticas, desofuscação, regras *YARA*, pontuação de risco (*risk scoring*), análise nativa com *Ghidra* quando necessário e geração de relatório), uma interface para navegador (*web*) em *React*, com assistente *Gemini* opcional, uma aplicação de secretária (*desktop*) *WPF* e um ambiente de execução isolado (*sandbox*) *Hyper-V* com dois caminhos de orquestração independentes (agente *HTTP* na máquina virtual (*vm-agent*) e *PowerShell Direct* com verificação de integridade *SHA-256*). A validação assenta em 220 testes automatizados, métricas substitutas de detecção (*proxy detection*) em 59 perfis sintéticos etiquetados estratificados por cinco níveis de severidade (precisão, sensibilidade (*recall*), F_1 e exatidão de nível 1,00), um conjunto benigno de oito executáveis *PE* compilados analisados estaticamente (especificidade 1,00, zero falsos positivos), metadados públicos de indicadores de compromisso (*IOC*) do *MalwareBazaar* (sem armazenamento de software malicioso (*malware*) na máquina anfitriã (*host*)), uma medição formal de desempenho (*benchmark*) estático (mediana 36,1 s, $n = 5$), perfis sintéticos de pontuação (*scoring*) alinhados com `test_risk_scoring.py` e execução benigna através de `benign-vm-test.exe` no Caminho B. O sistema foi concebido para explicabilidade e isolamento local, em detrimento de soluções comerciais de análise na nuvem (*cloud sandboxes*), destinando-se a uso académico e laboratorial.

Palavras-chave: análise de software malicioso (*malware*); troianos de acesso remoto (RAT); ambiente de execução isolado (*sandbox*); cibersegurança; engenharia de software; análise estática e dinâmica.

Acknowledgements

I thank my supervisor, **Professor João Bernardo Ferreira Sequeiros**, for his guidance, rigorous feedback, and consistent support throughout the development of this project. His suggestions on architecture, evaluation, and academic presentation helped me maintain a steady pace and refine each iteration of RAT ANALYZER before moving to the next milestone.

I also acknowledge the **open-source** community. Tools such as YARA, Ghidra, ILSpy, FastAPI, and React—together with the wider ecosystems that surround them—made it realistic to build an integrated static and dynamic malware analysis platform within the scope of a final-year project. Standing on maintained, well-documented software allowed this work to focus on system design and validation rather than reinventing foundational components.

Finally, I thank my family and friends for their encouragement during the most demanding stages of this work. Their patience when development consumed evenings and weekends, and their willingness to share honest opinions on the web interface and overall user experience, helped shape a tool that is not only technically sound but also approachable for students and analysts.

Contents

Abstract	i
Resumo	ii
Acknowledgements	iii
Contents	iv
List of Figures	vii
List of Tables	viii
Acronyms	x
1 Introduction	1
1.1 Context	1
1.2 Motivation	1
1.3 Objectives	2
1.4 Document Organisation	2
2 State of the Art	4
2.1 Background	4
2.2 Malware Analysis: Overview	4
2.2.1 Evolution of Malware Analysis	4
2.2.2 Static Versus Dynamic Detection: Challenges	5
2.2.3 Explainability Versus Machine Learning	5
2.2.4 Limitations of Current Approaches	5
2.2.5 Sandbox Virtualisation: Hyper-V, Docker, and KVM	6
2.2.6 Local Versus Cloud Analysis	6
2.3 Trends in RAT and Malware Detection	6
2.3.1 Commercial and Cloud Sandboxes	6
2.3.2 VirusTotal	7
2.3.3 Open-Source Frameworks	7
2.3.4 Machine Learning-Based Detection	7
2.4 Positioning Relative to Existing Solutions	7
2.5 Conclusions	8
3 Technologies and Tools Used	9
3.1 Introduction	9
3.2 Python and FastAPI	9
3.3 React, Vite and TypeScript	9
3.4 .NET 8, WPF and Minimal API	10

3.5	YARA, Ghidra and ILSpy	10
3.6	Hyper-V, PowerShell and SQLite	10
3.7	GitHub and LaTeX	10
3.8	Security and Isolation	11
3.9	Technology Choices and Rejected Alternatives	11
3.10	Conclusion	11
4	Software Engineering	12
4.1	Introduction	12
4.2	System Overview	12
4.3	Requirements Analysis	14
4.3.1	Functional Requirements	14
4.3.2	Non-Functional Requirements	16
4.3.3	Requirements Traceability	16
4.4	Diagrams	16
4.4.1	Use Case Diagram	18
4.4.2	Activity Diagram - Static Analysis	19
4.4.3	Activity Diagram - Dynamic Analysis	20
4.4.4	System Architecture	21
4.4.5	Trust Zones and Authentication	23
4.4.6	Sandbox Paths (A and B)	25
4.5	Security Architecture	27
4.6	Sandbox Path Comparison	28
4.7	Data Model	28
4.8	Conclusions	29
5	Implementation	30
5.1	Introduction	30
5.2	Backend - Static Analysis Pipeline	30
5.3	Web Frontend	31
5.3.1	Gemini Assistant in the Pseudo-C Panel	32
5.4	WPF Desktop Application	33
5.5	VM Agent and Hyper-V Sandbox	34
5.5.1	Path A - Backend + HTTP vm-agent	34
5.5.2	Path B - PowerShell Scripts	35
5.6	User Manual	36
5.7	Experimental Evaluation	36
5.7.1	Methodology and Ethical Considerations	36
5.7.2	Static Scoring Validation	37
5.7.3	Proxy Detection Metrics	37
5.7.4	Discussion of Evaluation Results	38
5.7.5	Benign Corpus Static Evaluation	39
5.7.6	Public IOC Metadata Reference	39
5.7.7	Dynamic Sandbox Validation	39
5.7.8	Formal Performance Benchmark	39
5.8	Tests	40

5.9	Conclusions	40
6	Conclusions and Future Work	41
6.1	Main Conclusions	41
6.2	Limitations	41
6.3	Future Work	42
A	Appendix	43
A.1	Token Verification in vm-agent	43
A.2	RunGate — Serialised Execution	43
A.3	Hyper-V Driver — Agent HTTP Client	44
A.4	VM Agent Limits	44
A.5	Report Transfer Protocol (Path B)	44
	Bibliography	46

List of Figures

4.1	End-to-end workflow (overview sequence)	13
4.2	High-level system pipeline	14
4.3	Use case diagram - RAT ANALYZER	18
4.4	Activity diagram - static analysis pipeline	19
4.5	Activity diagram - dynamic analysis pipeline	20
4.6	Conceptual architecture of RAT ANALYZER	21
4.7	Integrated sequence diagram - static, dynamic, and WPF flows . .	22
4.8	Trust zones - host / sandbox / guest isolation	23
4.9	Authentication map - tokens and credentials	24
4.10	Comparison of paths A (HTTP vm-agent) and B (Hyper-V PowerShell)	25
4.11	Sequence - Path A (HTTP vm-agent)	26
4.12	Sequence - Path B (PsDirect + SHA256 verification)	27
5.1	Web interface - upload zone (<i>Drop & Analyze</i>)	32
5.2	Web interface - report, pseudo-C/IL, and risk score	32
5.3	WPF desktop application - main screen	34
5.4	Hyper-V - VM MalwareSandbox and snapshot CleanState	36

List of Tables

2.1	Comparison of the RAT ANALYZER solution with existing alternatives	8
4.1	Functional requirements of RAT ANALYZER	15
4.2	Non-functional requirements of RAT ANALYZER	16
4.3	Risk score factors (0–100)	29
5.1	Static scoring validation scenarios	37
5.2	Proxy detection metrics on labelled synthetic profiles ($n = 59$) . . .	38
5.3	Benign corpus static evaluation ($n = 8$ PEs)	39
5.4	Formal benchmark - static pipeline and dynamic sandbox paths .	40
A.1	Execution controls in vm-agent (AgentLimits.cs, RunGate.cs) .	45

List of code excerpts

5.1	Post-transfer SHA-256 verification	35
A.1	Authentication middleware	43
A.2	Single-run gate	44
A.3	VM agent HTTP calls	44

Acronyms

API	Application Programming Interface
C2	Command and Control (command-and-control server)
CLI	Command Line Interface
DLL	Dynamic Link Library
ETW	Event Tracing for Windows
FR	Functional Requirement
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
IL	Intermediate Language (.NET)
JSON	JavaScript Object Notation
LLM	Large Language Model
NFR	Non-Functional Requirement
NDJSON	Newline-Delimited JSON (event streaming)
PE	Portable Executable
PsDirect	PowerShell Direct (remote access to Hyper-V guest)
RAT	Remote Access Trojan
REST	Representational State Transfer
RQ	Redis Queue
SQL	Structured Query Language
UBI	Universidade da Beira Interior
UUID	Universally Unique Identifier
VM	Virtual Machine
SHA-256	Secure Hash Algorithm 256 bits (file integrity)

WPF Windows Presentation Foundation

YARA Yet Another Recursive Acronym (detection rule language)

Chapter

1

Introduction

1.1 Context

This report presents the final-year project in Computer Science Engineering at Universidade da Beira Interior, dedicated to the development of RAT ANALYZER: an integrated static and dynamic analysis platform for detecting *Remote Access Trojans* (RATs) in Windows executables (.exe, .dll) and, with reduced scope, C# source files (.cs): on the web/API, .cs files undergo string-based heuristics only (no PE imports or ILSpy); compilation to .exe is available via the optional Tkinter GUI or manual build before upload.

RATs are among the most concerning classes of malware today: they allow an attacker to remotely control a compromised system, harvest credentials, exfiltrate data, and execute arbitrary commands. Their detection often requires combining *offline* techniques (static binary analysis) and *online* techniques (behaviour observation in a controlled environment), complemented by signature rules and scoring heuristics.

RAT ANALYZER was designed in an academic context as an **educational and triage** tool: in addition to flagging risk indicators, it exposes decompiled code, structured reports, and explanations of suspicious functions, enabling students and junior analysts to understand *why* a file is considered malicious. Complementarily, the web interface integrates a **Google Gemini**-based assistant that allows users to ask, in natural language, about the meaning of the pseudo-C excerpt visible in the results panel.

1.2 Motivation

The proliferation of malware with remote access capabilities demands accessible tools that integrate multiple analysis layers into a single workflow. Commercial solutions such as cloud sandboxes (VirusTotal, Any.Run, Hybrid

Analysis) are powerful but often unsuitable for teaching or local laboratory contexts, whether due to cost or sample confidentiality.

This project was motivated by the need to **integrate** static analysis, YARA rules, decompilation, and sandbox execution in a single workflow spanning an API, a web interface, and a desktop application. Equally important is **isolation**: potentially malicious samples must run inside a Hyper-V virtual machine without Internet connectivity, with automatic restore to a clean snapshot after each run. The platform must also favour **transparency**, producing detailed reports—imports, C2 strings, evasion indicators, risk score, pseudo-C/IL—rather than a binary verdict alone, with optional natural-language explanation of decompiled excerpts. Finally, **academic reproducibility** requires open-source code, versioned documentation, automated tests, and security guidance suitable for supervised laboratory deployment.

1.3 Objectives

The overarching goal of RAT ANALYZER is to deliver an integrated static and dynamic analysis platform for RAT triage in an academic setting: the analyst uploads a suspect file, receives an explainable risk assessment grounded in concrete indicators, and may optionally observe behaviour in an isolated sandbox before deciding on further action.

To realise this goal, the project pursued several concrete aims. The system provides a Python **backend** that receives samples, executes a complete static analysis pipeline, and orchestrates dynamic analysis jobs in static, dynamic, or combined modes. A **web interface** offers drag-and-drop upload, mode selection, progress streaming, report visualisation, decompiled code panels, an optional Gemini assistant, and bilingual support (Portuguese/English). A **WPF desktop application** serves as an alternative entry point with dependency bootstrap, persistent language selection, and Hyper-V sandbox integration via Path B. A **VM agent** and PowerShell automation enable isolated sample execution and behavioural telemetry collection inside the guest. An **explainable scoring model** (0–100) aggregates YARA matches, C2 indicators, evasion techniques, packers, obfuscation, and persistence signals. Finally, the solution is **validated** through automated tests, benign samples, and operational documentation for local deployment.

1.4 Document Organisation

This report comprises six chapters, an appendix, and a bibliography. **Chapter 2** (*State of the Art*) surveys malware analysis techniques, commercial and open-

source alternatives, and positions RAT ANALYZER relative to existing sandboxes and detection approaches. **Chapter 3** (*Technologies and Tools Used*) describes the multi-language stack—Python/FastAPI, React, .NET/WPF, YARA, Ghidra, IL-Spy, Hyper-V, and SQLite—together with security and isolation choices. **Chapter 4** (*Software Engineering*) presents functional and non-functional requirements, UML diagrams, the data model, trust zones, and the two sandbox orchestration paths. **Chapter 5** (*Implementation*) details each software component, the user manual, experimental evaluation, and the automated test suite. **Chapter 6** (*Conclusions and Future Work*) summarises achievements, acknowledges limitations, and outlines extensions such as multi-VM scaling, Linux support, and MITRE ATT&CK mapping. The **appendix** reproduces representative source excerpts for the vm-agent and Hyper-V driver. The **bibliography** lists the references cited throughout the document.

Chapter

2

State of the Art

2.1 Background

This chapter frames malware analysis techniques for RAT detection and compares widely used static, dynamic, and hybrid solutions with RAT ANALYZER.

2.2 Malware Analysis: Overview

Malware analysis is commonly divided into two complementary approaches (1; 2; 16; 17). **Static analysis** examines the file without executing it: PE structure, sections, imports/exports, strings, entropy, YARA signatures, decompilation, and detection of evasion techniques (anti-debug, anti-VM). **Dynamic analysis** runs the sample in an isolated sandbox and monitors behaviour—process creation, registry changes, files, network activity, and persistence.

In the RAT context, static analysis identifies suspicious imports and C2 strings without execution risk; dynamic analysis reveals connections, files, and persistence behaviour. Neither approach is sufficient alone: packers and polymorphism defeat naive static signatures, while sandbox evasion (anti-VM, sleep delays, environment checks) can suppress dynamic indicators (16; 17; 30).

The subsections below trace how these techniques evolved, where they fail in practice, and why RAT ANALYZER favours a hybrid, explainable design.

2.2.1 Evolution of Malware Analysis

Malware analysis has evolved from manual disassembly and signature matching in the late 1990s toward automated pipelines that combine sandbox execution, rule engines, and machine learning (1; 18; 19). Early dynamic tools such as CWSandbox and TTAalyze demonstrated that behavioural traces could complement antivirus heuristics, but required carefully controlled environments and human interpretation of noisy logs. The last decade brought cloud multi-scanner services, open-source sandboxes (Cuckoo, CAPE), and deep-

learning classifiers on PE features (12; 20; 27; 29). Contemporary RAT families (AsyncRAT, QuasarRAT, Remcos) further stress this landscape: they blend legitimate remote-administration APIs with obfuscation, staged payloads, and commodity packers, making triage a *multi-evidence* problem rather than a single score.

2.2.2 Static Versus Dynamic Detection: Challenges

Static analysis is fast, repeatable, and safe—no execution on the analyst host—but struggles with encrypted payloads, runtime decryption, and novel variants without known strings or imports. **Dynamic analysis** observes ground-truth behaviour yet incurs higher latency, risks sandbox escape if isolation fails, and may miss logic that activates only under specific user interaction or after long delays (2; 17; 26). Hybrid workflows therefore remain the professional norm: static triage narrows the candidate set; dynamic confirmation validates suspicious indicators (30). RAT ANALYZER adopts this hybrid model explicitly, prioritising explainable static signals before optional sandbox execution.

2.2.3 Explainability Versus Machine Learning

Machine-learning detectors on raw bytes or PE feature vectors achieve strong benchmark metrics (12; 13; 21; 27) but often function as opaque classifiers: an analyst cannot easily justify a verdict to a student, auditor, or court. Recent work on explainable behavioural detection (25) and interpretable Windows PE models (27; 28) highlights this tension. In educational and laboratory settings, **interpretability** matters as much as accuracy. Rule-based YARA matches, scored heuristics (C2 strings, persistence, evasion), and decompiled pseudo-C provide auditable evidence chains. Large language models offer a complementary layer—natural-language explanation of code excerpts—without replacing the underlying indicators. RAT ANALYZER deliberately favours this *glass-box* design over end-to-end neural classification, accepting that novel malware may score below threshold until rules or heuristics are updated.

2.2.4 Limitations of Current Approaches

No single commercial or open-source platform eliminates false positives and false negatives. Multi-AV aggregation (VirusTotal) improves coverage but shares samples externally and hides engine reasoning. Cloud sandboxes (Any.Run, Hybrid Analysis, Joe Sandbox) excel at rich telemetry yet impose cost, latency, and confidentiality constraints for universities holding sensitive student or research data. Self-hosted sandboxes (Cuckoo/CAPE) demand substantial operational expertise. ML models require labelled datasets that are expensive

to curate and drift as families evolve (24; 23; 28). These gaps motivate a **local**, **transparent**, and **modular** platform rather than another opaque cloud service.

2.2.5 Sandbox Virtualisation: Hyper-V, Docker, and KVM

Container platforms such as Docker are ill-suited to faithful Windows malware behaviour: many RATs depend on Win32 APIs, GUI subsystems, registry persistence, and process injection patterns that do not translate cleanly to Linux containers or minimal Windows images. **KVM/QEMU** offers full hardware virtualisation and is common in Linux-centric research labs, but adds integration overhead on Windows workstations where Hyper-V is already enabled for development. **Hyper-V** provides type-1 hypervisor isolation on laboratory PCs, snapshot restore for repeatable runs, and native PowerShell automation—trade-offs favouring a Windows-native academic deployment over cross-platform portability. Docker and KVM remain valid alternatives where the host OS or curriculum differs; they were evaluated and deferred (Section 3).

2.2.6 Local Versus Cloud Analysis

Submitting samples to public sandboxes accelerates triage but leaks file content to third parties and prevents custom rule development inside the analysis environment. A **local sandbox** keeps specimens on institutional infrastructure, supports offline teaching, and allows students to inspect decompilation and scoring logic without account quotas. The cost is operational: the analyst maintains VMs, snapshots, and updates. Configuration choices materially affect what a sandbox observes (26). RAT ANALYZER targets this local model consciously, accepting higher per-run latency than commercial clouds in exchange for confidentiality, reproducibility, and pedagogical depth.

2.3 Trends in RAT and Malware Detection

Beyond foundational static and dynamic methods, several product categories shape how analysts detect RATs today. The following subsections review commercial sandboxes, multi-engine scanners, open-source frameworks, and machine-learning approaches, highlighting trade-offs that informed the design of RAT ANALYZER.

2.3.1 Commercial and Cloud Sandboxes

Platforms such as *Any.Run* (3), *Hybrid Analysis* (4), and *Joe Sandbox* (5) offer rich dynamic analysis (18; 19; 26), but impose cost, external sample submis-

sion, and limited environment customisation—constraints that RAT ANALYZER addresses with local, isolated, zero-cost execution.

2.3.2 VirusTotal

VirusTotal (6) aggregates many antivirus engines for triage, but shares submitted samples with partners and does not expose decompiled code or custom detection rules.

2.3.3 Open-Source Frameworks

Several open-source tools address parts of the malware-analysis pipeline, though none combine all capabilities targeted by RAT ANALYZER in a single educational workflow. **Cuckoo Sandbox** (7) is a self-hostable, extensible Python sandbox with behavioural analysis on Windows and Linux; it remains the open-source reference closest to RAT ANALYZER, but requires complex configuration and does not integrate decompilation or explainable scoring. **CAPE Sandbox** (8), a Cuckoo fork, specialises in extracting configurations from malware families and C2 servers—highly valuable, yet too narrow for generalist teaching. **YARA** (9) provides the rule language used across the industry; RAT ANALYZER embeds YARA as one phase of the static pipeline alongside scoring and decompilation. **Ghidra** and **ILSpy** (10; 11) are essential decompilers typically used from the command line; RAT ANALYZER orchestrates both and exposes results in the web interface.

2.3.4 Machine Learning-Based Detection

Deep learning on PE features achieves high classification metrics (12; 13; 20; 21; 29) but yields opaque decisions. RAT ANALYZER prioritises **explainability** via concrete indicators and an optional LLM assistant, aligning with recent calls for transparent behavioural and static models (25; 27). Public IOC repositories such as MalwareBazaar (22) and labelling tools such as AVclass (23) complement local triage (24).

2.4 Positioning Relative to Existing Solutions

RAT ANALYZER combines multi-phase static analysis with a native Hyper-V sandbox and integrated decompilation, exposed through web and WPF interfaces. Table 2.1 summarises the positioning.

Table 2.1: Comparison of the RAT ANALYZER solution with existing alternatives

Reference	Technology	RAT ANALYZER solution
VirusTotal	Cloud multi-AV	Local; YARA + heuristics + Hyper-V sandbox
Cuckoo/CAPE	Self-hostable sandbox	Native Windows sandbox (Hyper-V)
Any.Run / Hybrid	Commercial sandbox	Free; educational focus; pseudo-C/IL
YARA standalone	Signature rules	YARA + scoring + decompilation
Ghidra/ILSpy CLI	Offline decompilation	Orchestrated decompilation + web UI
ML classifiers	Opaque models	Explainable indicators + report + optional LLM assistant

2.5 Conclusions

Effective RAT detection benefits from multi-layer analysis that combines static indicators, controlled execution, and human-interpretable reporting. The literature shows a clear tension between scalable opaque ML and explainable rule-based triage; cloud sandboxes and local hypervisor sandboxes serve different institutional needs. RAT ANALYZER positions itself for **local academic use**: Hyper-V isolation, decompilation orchestration, scored heuristics, and optional LLM-assisted code explanation, rather than commercial-scale multi-tenant analysis or black-box classification alone.

Chapter

3

Technologies and Tools Used

3.1 Introduction

This chapter summarises the technologies used in RAT ANALYZER: backend, interfaces, sandbox, and report.

3.2 Python and FastAPI

Python 3.10+ was adopted as the main backend language, due to its maturity in the information security ecosystem and the wide availability of libraries for PE file analysis, binary format parsing, and integration with decompilation tools. **FastAPI** (15) was chosen as the web framework for providing automatic data validation with Pydantic, interactive OpenAPI documentation (/docs), and native support for asynchronous NDJSON streaming (/api/analyze_stream), essential for real-time feedback during analysis. There is also an optional **Tkinter** graphical interface for local static analysis without a browser, useful for quick demonstrations.

3.3 React, Vite and TypeScript

The web interface was built with **React 18** and **TypeScript**, ensuring static typing that reduces errors during development. The **Vite** bundler enables instant hot-reload during development and optimised production builds. For visual styling, **Tailwind CSS** and **shadcn/ui** were adopted; transition animations are handled with **Framer Motion**. The unit test suite uses **Vitest** (62 unit tests). Integration with **Google Gemini** (gemini-2.0-flash) is done directly in the browser: the user's API key is stored in `localStorage` and never passes through the FastAPI backend, ensuring the server has no access to third-party credentials.

3.4 .NET 8, WPF and Minimal API

Three .NET 8 projects share the `RatAnalyzer.sln` solution: **Rat Analyzer** (WPF desktop, compiles to `Rat Analyzer.exe`), **VmAgent** (Minimal API running inside the sandbox VM to receive and execute samples), and **Benign-VmTest** (benign executable to validate the dynamic pipeline without risk). WPF serves as an alternative entry point to the web interface, with automatic dependency bootstrap, persistent language selection, and direct integration with the Hyper-V sandbox (Path B).

3.5 YARA, Ghidra and ILSpy

YARA (9) is the pattern-matching rule language used for signature-based detection. Rules reside in `yara_rules/` and are compiled and applied to the binary under analysis. **ILSpy CLI** (11) decompiles .NET assemblies into readable C# code, enabling inspection of executables written with the managed runtime. **Ghidra 12+** (10), developed by the NSA, is used for decompilation of native PE binaries, producing pseudo-C that is exposed in the web interface code panel.

3.6 Hyper-V, PowerShell and SQLite

The dynamic sandbox relies on **Hyper-V**, the native Windows hypervisor, with a VM named `MalwareSandbox`, connected to an **Internal** switch with no Internet connectivity on the guest and a `CleanState` snapshot that is restored before each analysis. **PowerShell** automates one-time VM setup (`01-Setup-MalwareSandbox.ps1`) and, on **Path B**, sample copy, guest execution, and report transfer (`04-Run-Sample.ps1`). On **Path A**, the Python backend (`vm_drivers/hyperv.py`) uses PowerShell only for snapshot restore and VM start; sample upload, execution, and report retrieval go through the HTTP vm-agent. **SQLite** persists analysis jobs in `analysis.db`; **Redis** with **RQ** is a configurable option for distributed queuing in environments with multiple workers.

The backend uses a pluggable sandbox driver so that VM orchestration can change between development, Hyper-V, or future Proxmox deployments without rewriting the job pipeline. The orchestrator calls the active driver for restore, execution, and report retrieval while keeping the analysis contract unchanged.

3.7 GitHub and LaTeX

GitHub hosts the project repository, with Git version control and continuous integration via **GitHub Actions** (CI with `pytest`, `Vitest`, and `dotnet build`). Changes are recorded in `CHANGELOG.md` and development conventions in the

root README.md. This report was written in ~~La~~^{TeX}, a typesetting system widely used in academic and scientific contexts, which automatically manages section numbering, figures, tables, and bibliographic references with rigour and consistency.

3.8 Security and Isolation

The design adopts *fail-closed* in production: the backend refuses to start without `RATANALYZER_API_TOKEN` when `RATANALYZER_ENV=production`; the vm-agent refuses to start without `VM_AGENT_TOKEN`, except when `VM_AGENT_ALLOW_INSECURE=1` in local development.

Secrets reside in `secrets/` (gitignored) and are produced by `scripts/generate-production-secrets.ps1`. HTTP uploads require authentication, with name sanitisation, size limits, and storage in `DATA_DIR` outside the repository. The canonical security documentation is in `docs/SEGURANCA.md`.

3.9 Technology Choices and Rejected Alternatives

Major stack decisions balanced **local reproducibility**, **Windows fidelity**, and **academic maintainability**. Hyper-V with snapshot restore was preferred over Docker because RAT behaviour depends on full Win32 semantics; Docker excels at service isolation but poorly replicates GUI, registry, and injection patterns relevant to this project. KVM offers comparable isolation on Linux hosts but would have forced a heterogeneous lab setup on predominantly Windows workstations. FastAPI was chosen over Flask or Django for asynchronous streaming and schema validation; React over alternative SPA frameworks for ecosystem familiarity and test tooling; WPF over Electron to avoid bundling a browser runtime while retaining direct Hyper-V integration. Redis-backed queuing remains optional so the platform operates offline with local worker threads. The Gemini assistant runs in the browser only: decompiled excerpts and API keys stay on the client, which protects sample confidentiality at the cost of centralised LLM control.

3.10 Conclusion

The stack reflects the heterogeneous nature of malware analysis: Python for pipeline and API, TypeScript for UX, C# for Windows/Hyper-V integration, and PowerShell for automation.

Chapter

4

Software Engineering

4.1 Introduction

This chapter details system architecture and design: functional and non-functional requirements, diagrams, data model, and security architecture.

4.2 System Overview

The following figures summarise the end-to-end workflow and high-level architecture; the diagram subsections below refine each element.

Figure 4.1 situates the analyst workflow: a sample enters through the web or desktop interface, undergoes static analysis on the host, and may optionally be executed in an isolated Hyper-V sandbox. Results converge in a structured report with decompiled code and an explainable risk score.

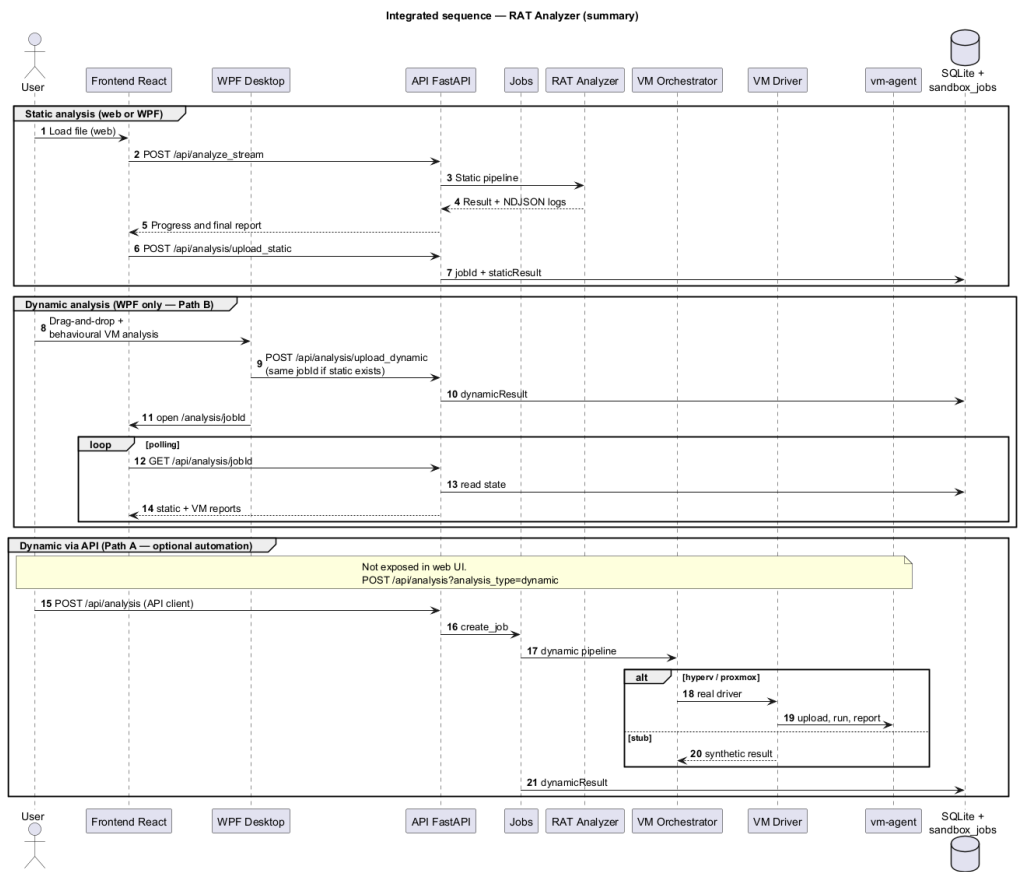


Figure 4.1: End-to-end workflow—sample submission, static analysis, optional sandbox execution, and unified reporting (overview sequence)

Figure 4.2 shows how the presentation tier, API, analysis engine, and sandbox layers relate at architectural level.

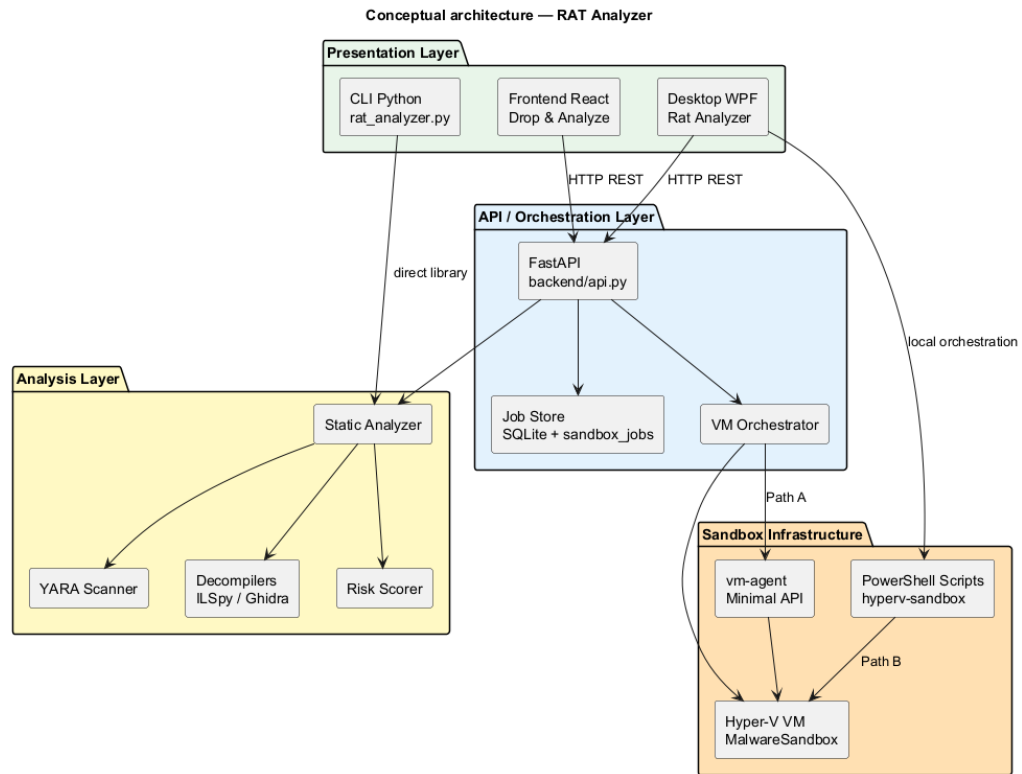


Figure 4.2: High-level system pipeline—presentation, API, analysis engine, and sandbox isolation

4.3 Requirements Analysis

4.3.1 Functional Requirements

Functional requirements were elicited from the intended analyst workflow, the need for isolated dynamic execution, and the academic goal of explainable triage rather than opaque classification. Stakeholder interviews with the project supervisor, together with comparison against commercial sandboxes and open-source frameworks (Chapter 2), informed the scope of upload validation, analysis modes, and reporting features. Table 4.1 presents the functional requirements identified for RAT ANALYZER.

Table 4.1: Functional requirements of RAT ANALYZER

ID	Description
FR 01	The system shall accept sample uploads in .exe, .dll, and .cs formats via the web interface, desktop application, or REST API. The system shall validate file extension, enforce a configurable size limit (default 100 MB), and sanitise file names. For .cs submissions on the web/API, the system shall apply string-based heuristics only; full PE analysis shall require a compiled binary.
FR 02	The system shall execute a seven-phase static analysis pipeline comprising PE metadata extraction, managed decompilation when applicable, import/string/evasion heuristics, deobfuscation, YARA scanning, risk scoring, native decompilation when required, and structured report generation.
FR 03	The system shall execute samples in an isolated Hyper-V virtual machine without Internet connectivity and return a behavioural report. The user shall select static, dynamic, or combined analysis modes; the web interface shall use Path A and the WPF application shall use Path B.
FR 04	The system shall create, query, and list analysis jobs identified by UUID. The system shall persist job metadata in SQLite and store per-job artefacts.
FR 05	The system shall present analysis results in a web interface with report, pseudo-C, and IL panels. The user shall navigate cross-references between panels. The system shall provide a permalink with live status updates during sandbox runs.
FR 06	The system shall compute a risk score from 0 to 100 by aggregating YARA matches, C2 indicators, evasion techniques, packers, obfuscation, persistence signals, and entropy. The system shall assign one of five severity levels and apply strong-evidence gates for the highest tiers.
FR 07	The system shall stream incremental NDJSON log events during static analysis so that the user receives real-time progress feedback.
FR 08	The system shall orchestrate dynamic analysis through two paths: (A) a backend-driven HTTP agent inside the guest, and (B) PowerShell Direct with post-transfer integrity verification from the WPF client. Path B shall publish sandbox results to the same job as a prior static run when applicable.
FR 09	The system shall offer an optional browser-side assistant that explains visible decompiled pseudo-C excerpts. The user's Gemini API key shall remain in local browser storage. The system shall provide a demonstration mode that does not contact external services.

4.3.2 Non-Functional Requirements

Non-functional requirements express quality attributes essential to malware analysis in a laboratory environment: confidentiality of samples, isolation of execution, usability for students, and maintainability for long-term academic use. Table 4.2 lists the non-functional requirements alongside their rationale.

Table 4.2: Non-functional requirements of RAT ANALYZER

ID	Description
NFR 01	The system shall bind the API to localhost in production deployments. The system shall require authentication tokens before serving analysis requests. Secrets shall not be stored in version control.
NFR 02	The system shall connect the sandbox virtual machine to an internal switch without Internet routing. The system shall restore a clean snapshot after each dynamic run. Path B shall verify network isolation before sample execution.
NFR 03	The system shall support drag-and-drop upload, real-time progress display, and a responsive layout. A user with basic security knowledge shall be able to interpret results without formal training.
NFR 04	The system shall support at least two concurrent analysis workers by default. The administrator shall configure execution and VM-operation timeouts per sandbox path.
NFR 05	The system shall be verifiable through automated continuous integration (pytest, Vitest, dotnet build). Dependencies shall be locked and technical documentation versioned.
NFR 06	The system shall operate without an external Redis instance by falling back to local worker threads. An optional Redis queue shall be available for distributed deployments.

4.3.3 Requirements Traceability

Each functional requirement maps to a primary component (backend API, static pipeline, sandbox, job store, web UI, scoring module, streaming API, VM orchestration, Gemini assistant) and is verified by automated tests in continuous integration. The full traceability matrix, listing modules and test suites per requirement, is omitted here to avoid duplicating repository structure; coverage is demonstrated by the 220 automated tests summarised in Section 5.8.

4.4 Diagrams

PlantUML diagrams (synchronised via `sync_diagrams_to_report.py`) document functional flows, architecture, security boundaries, and sandbox orches-

tration. Each subsection below introduces one diagram and explains its role in the design; the figures are referenced throughout Chapters 4 and 5.

4.4.1 Use Case Diagram

Figure 4.3 identifies the primary actors—analyst, backend services, sandbox guest, and optional LLM assistant—and the use cases they trigger, from sample upload through static analysis, dynamic execution, and report consultation.

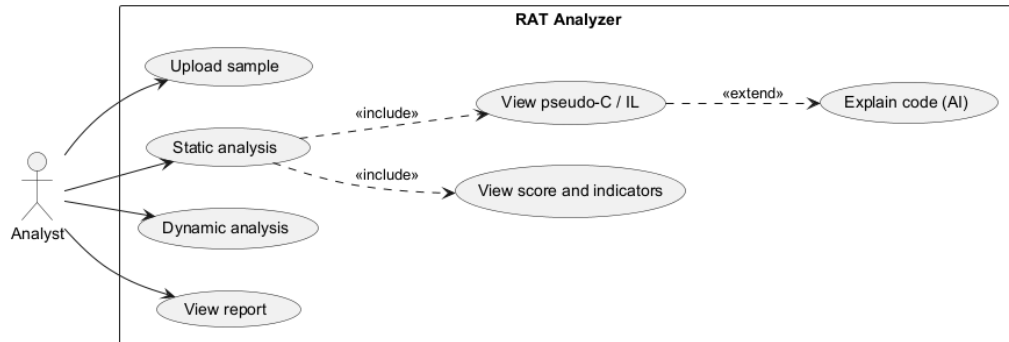


Figure 4.3: Use case diagram - RAT ANALYZER

4.4.2 Activity Diagram - Static Analysis

Figure 4.4 details the seven-phase static pipeline executed on the host: meta-data extraction, managed and native decompilation branches, heuristic scanning, YARA matching, risk scoring, and report assembly.

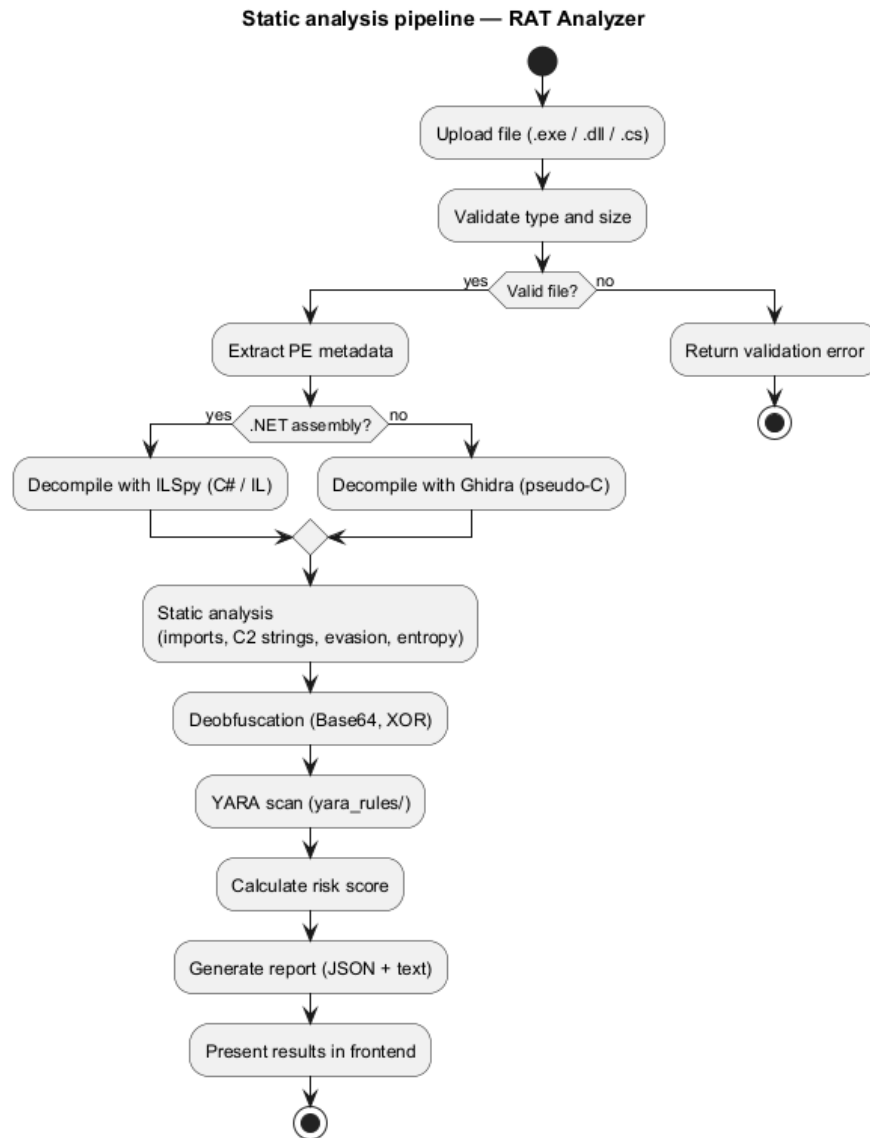


Figure 4.4: Activity diagram - static analysis pipeline

4.4.3 Activity Diagram - Dynamic Analysis

Figure 4.5 shows the dynamic path from job submission through snapshot restore, guest execution, telemetry collection, and report transfer back to the host.

Dynamic analysis pipeline — RAT Analyzer

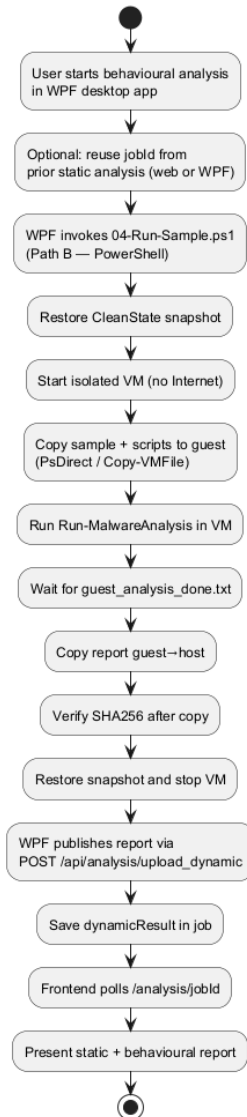


Figure 4.5: Activity diagram - dynamic analysis pipeline

4.4.4 System Architecture

Figure 4.6 presents the conceptual layering of RAT ANALYZER: presentation tier (web and WPF), FastAPI backend, analysis modules, job store, and Hyper-V sandbox.

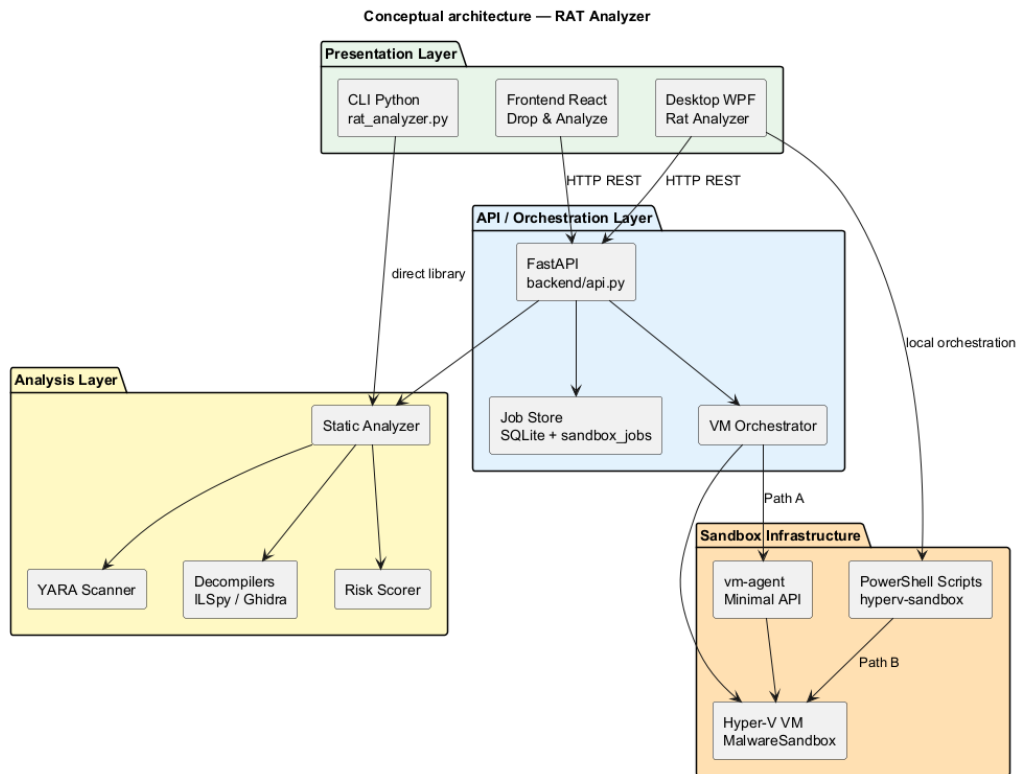


Figure 4.6: Conceptual architecture of RAT ANALYZER

Figure 4.7 extends this view with an integrated sequence diagram covering static, dynamic, and WPF-initiated flows on a single timeline.

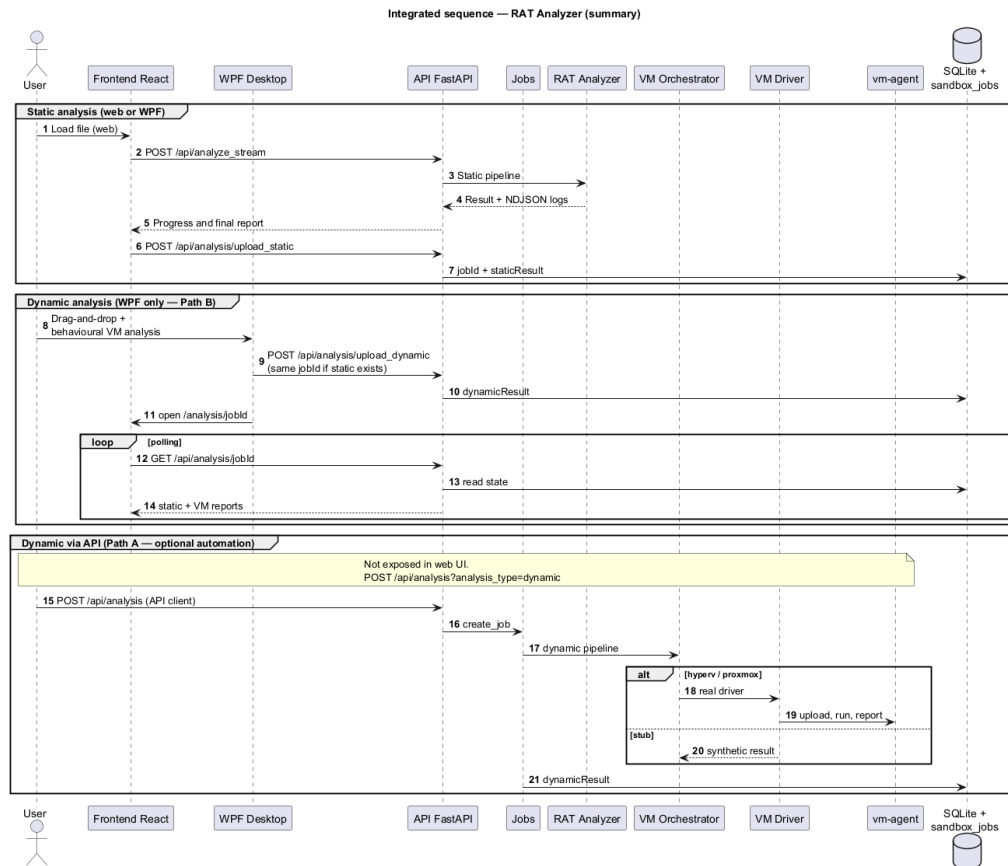


Figure 4.7: Integrated sequence diagram - static, dynamic, and WPF flows

4.4.5 Trust Zones and Authentication

Figure 4.8 maps the three trust zones—host, isolated sandbox network, and untrusted guest—and the data flows crossing each boundary.

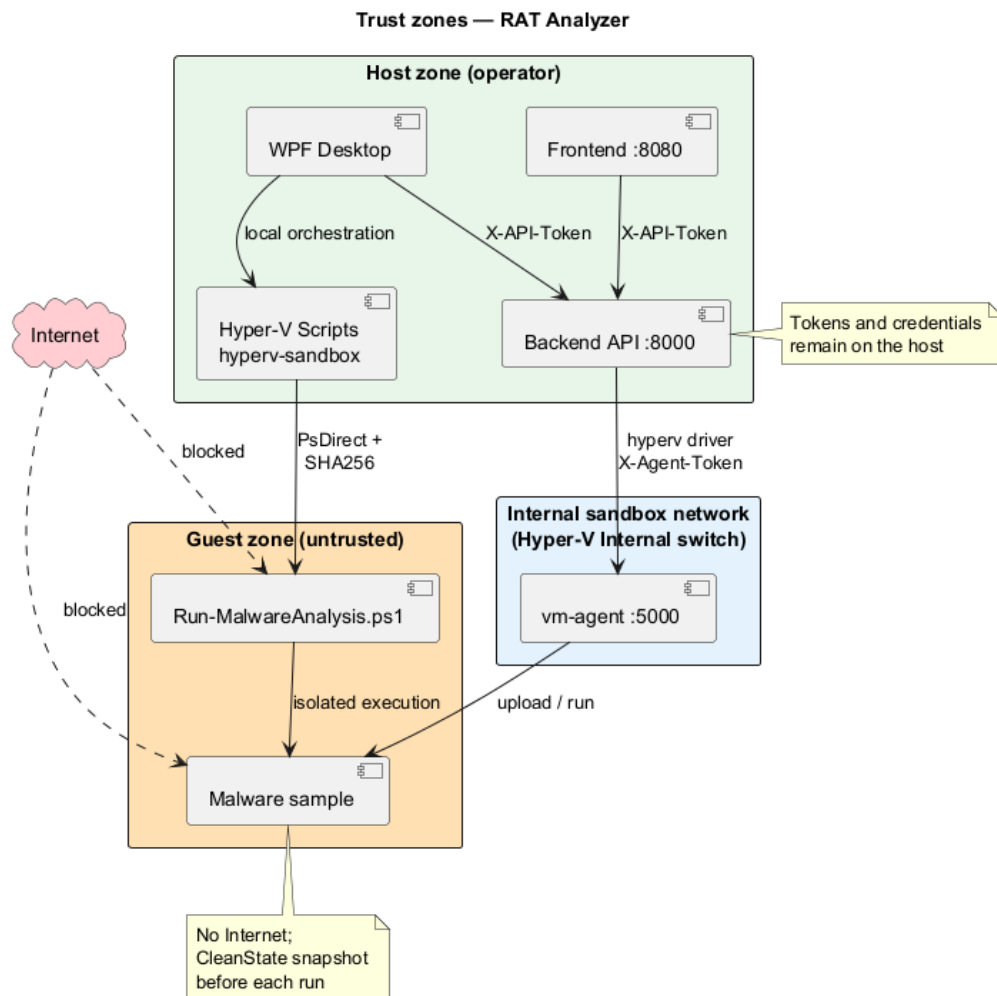


Figure 4.8: Trust zones - host / sandbox / guest isolation

Figure 4.9 complements the trust-zone view with the authentication tokens and credentials required by each component.

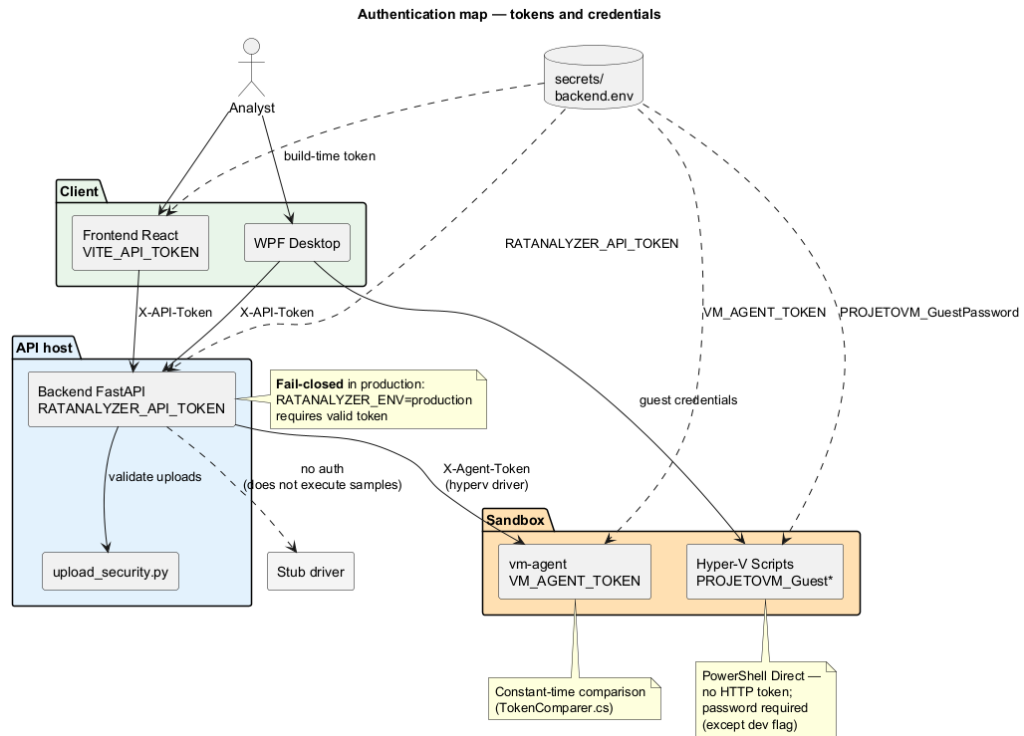


Figure 4.9: Authentication map - tokens and credentials

4.4.6 Sandbox Paths (A and B)

Figure 4.10 compares the two dynamic orchestration paths at a glance.

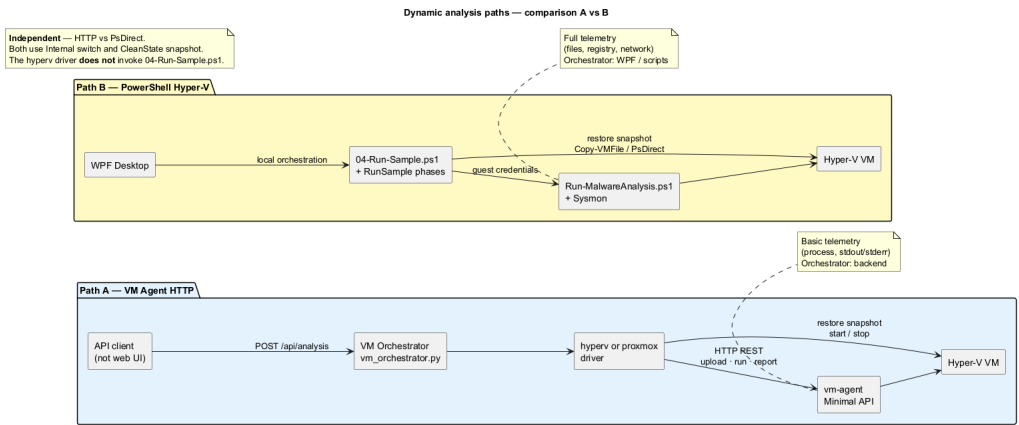


Figure 4.10: Comparison of paths A (HTTP vm-agent) and B (Hyper-V PowerShell)

Figure 4.11 sequences the HTTP vm-agent interactions on Path A: snapshot restore, sample upload, execution, and report retrieval.

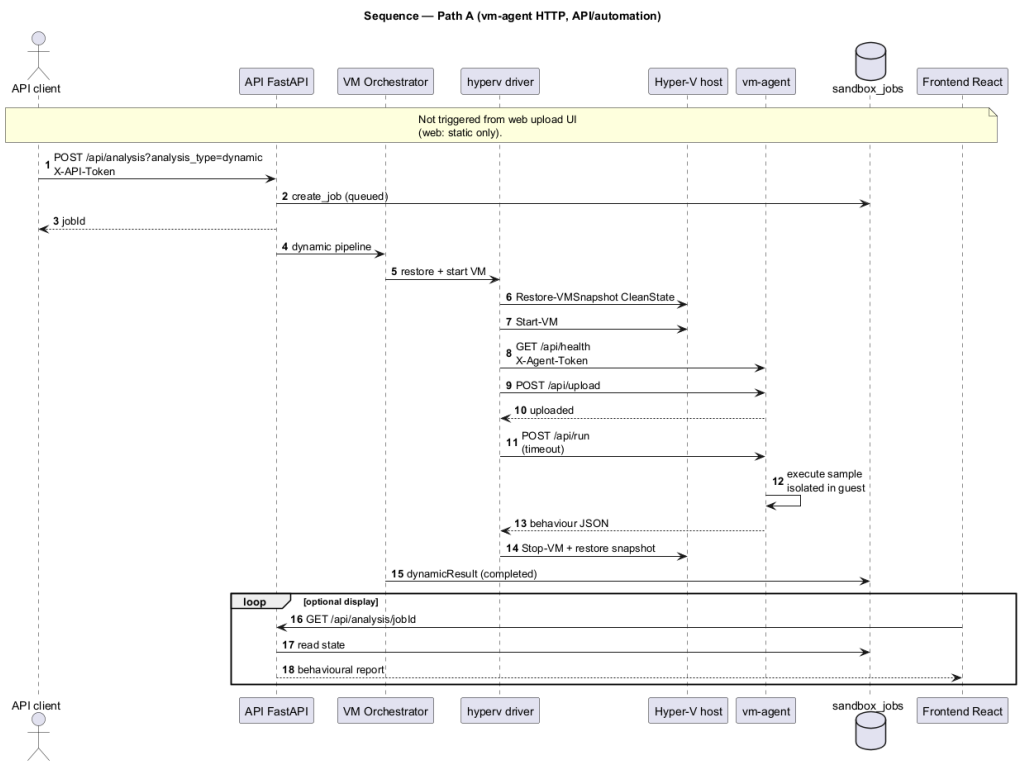


Figure 4.11: Sequence - Path A (HTTP vm-agent)

Figure 4.12 details the PowerShell Direct workflow on Path B, including SHA-256 integrity verification of the report copied from the guest.

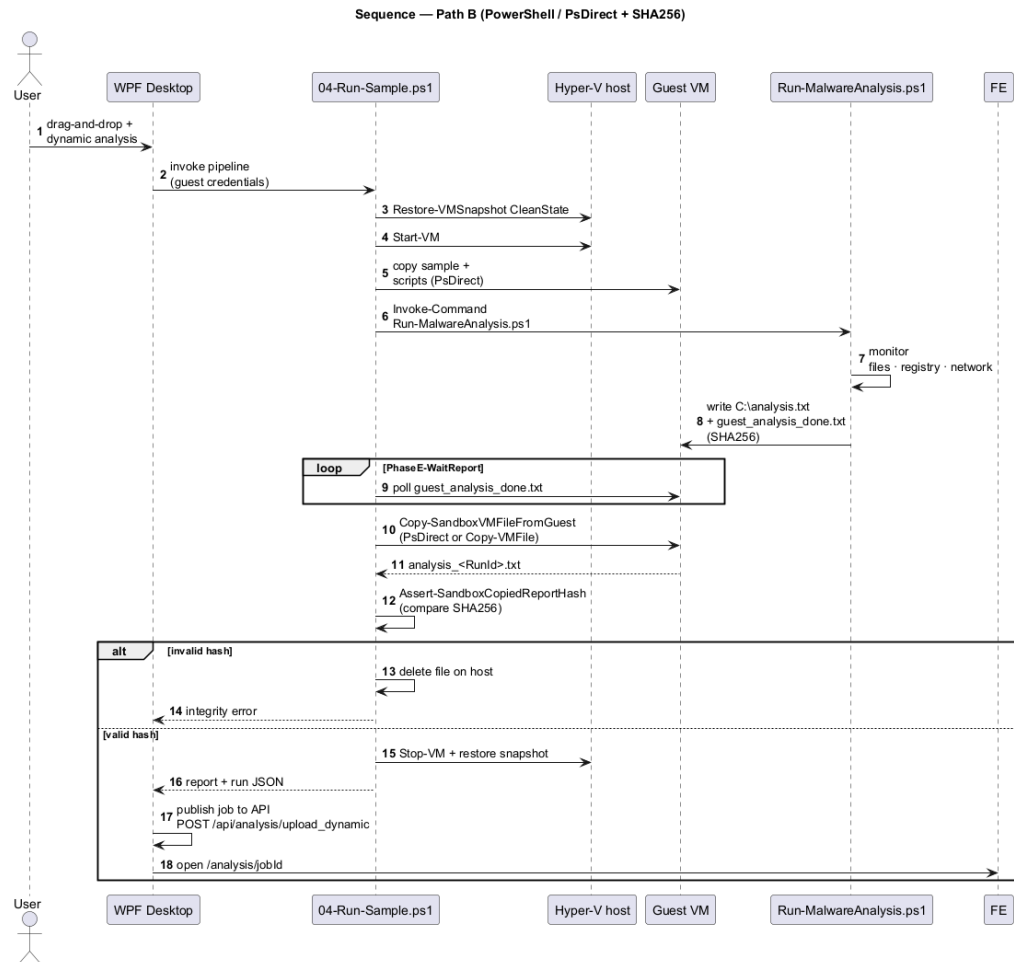


Figure 4.12: Sequence - Path B (PsDirect + SHA256 verification)

4.5 Security Architecture

Three trust zones apply: **host** (trusted analysis services), **sandbox network** (isolated internal switch), and **guest** (untrusted sample execution). The design follows a *fail-closed* policy: production services refuse to start without configured authentication tokens; malware executes only inside the VM or a non-executing stub driver; secrets remain outside version control; the API listens on localhost; and guest accounts operate with least privilege while host administrators manage Hyper-V lifecycle. Defence in depth combines

token authentication, upload rate limiting, name sanitisation, size caps, and mandatory snapshot restore between runs.

Authentication is layered by component: the backend and WPF client share an API token; the in-guest agent validates a separate agent token; Hyper-V automation uses PowerShell Direct with dedicated guest credentials. The optional Gemini integration never receives backend credentials—the user’s API key stays in browser storage—reducing the attack surface for third-party LLM providers. Primary residual risks include credential exposure in PowerShell logs (mitigated by environment variables and secure dialogs), optional client-side code exfiltration to Gemini (user-controlled), and dependency tampering (mitigated by hash-locked installs and optional binary verification on WPF startup). Figures 4.8 and 4.9 illustrate these zones and token flows; detailed endpoint and variable listings appear in the appendix for operators.

4.6 Sandbox Path Comparison

Paths A and B address the same isolation goal through different orchestration trade-offs (Figure 4.10). **Path A** integrates dynamic analysis into the Python backend. After snapshot restore, the host talks to an HTTP agent inside the guest for upload, execution, and report retrieval. This path suits the web workflow but collects lighter telemetry. **Path B** is driven by the WPF desktop and PowerShell. Samples reach the guest via PowerShell Direct or VM file copy; a guest-side collector gathers richer diffs (files, registry, network, Sysmon when present); and the host verifies report integrity with SHA-256 before accepting results. Recent sandbox systematisation work (26) stresses that monitoring depth and configuration strongly affect observable behaviour—a finding that motivated keeping Path B as a deeper, independently evolvable instrumentation path. Both paths share Hyper-V, the internal switch, and clean-snapshot restore.

4.7 Data Model

Persistence relies on **SQLite** with one record per analysis job (UUID identifier, analysis type, status, file metadata, timestamps). Per-job artefacts include static and dynamic reports, deobfuscation excerpts, and an optional sample copy. Jobs progress through four states—*queued*, *running*, *completed*, and *failed*—enabling the web interface to poll status during long sandbox runs.

Table 4.3 summarises the maximum caps per factor in the risk scorer (each factor contributes up to its respective cap; the final score is limited to 100).

Table 4.3: Risk score factors (0–100)

Factor	Cap	Factor	Cap
C2 strings	20	Persistence	10
YARA matches	25	Packer	10
Suspicious functions	20	Obfuscation	8
Suspicious imports	12	High entropy	5
Stealer indicators	15		
Evasion techniques	12		

Levels: CRITICAL if score ≥ 72 and `strong_evidence` (YARA match, or strong C2 combined with suspicious functions/imports); HIGH if score ≥ 48 with at least one strong signal; MEDIUM if score ≥ 35 ; LOW if score ≥ 15 ; otherwise VERY LOW. (The production API returns Portuguese labels: CRÍTICO, ALTO, MÉDIO, BAIXO, MUITO BAIXO.)

4.8 Conclusions

Modelling separates presentation, API, analysis, and sandbox concerns. Requirements remain traceable through automated CI coverage rather than exhaustive in-report module listings.

Chapter

5

Implementation

5.1 Introduction

This chapter describes the implemented system and explains the reasoning behind major design choices. For each component, two perspectives are presented: **functional behaviour** (what the system does) and **design rationale** (why that approach was adopted and which trade-offs it entails). Architectural context and diagrams appear in Chapters 4 and 2; validation interpretation appears in Section 5.7.4.

The primary use case—an analyst submitting a suspicious Windows executable—proceeds as follows: the system validates the upload, assigns a unique job identifier, runs static analysis (with optional real-time streaming on the web), optionally schedules sandbox execution, and presents unified results with decompiled code, behavioural findings, and an explainable risk score. The analyst may optionally query the Gemini assistant about suspicious pseudo-C excerpts.

5.2 Backend - Static Analysis Pipeline

Functional behaviour. The static analysis engine orchestrates seven phases: (1) PE metadata extraction; (2) managed decompilation via ILSpy when the sample is a .NET assembly; (3) import, string, and evasion heuristics; (4) deobfuscation of common encodings; (5) YARA rule matching; (6) aggregation into a 0–100 risk score with five severity levels; and (7) structured report generation, including native disassembly and Ghidra pseudo-C when ILSpy cannot produce managed code. Phases run sequentially on the host; results are persisted per job and exposed through a REST API that supports synchronous analysis, NDJSON streaming for live progress, asynchronous job creation for combined static/dynamic modes, artefact retrieval, and storage management. Web uploads of .cs source files receive string heuristics only; full PE analysis requires a compiled binary.

Design rationale. Phases are ordered *cheap-before-expensive* because YARA compilation and Ghidra dominate wall-clock time on typical laboratory hardware (Table 5.4). Running lightweight checks first lets the pipeline fail fast on obvious benign samples and surface early indicators before committing to decompilation—a pattern aligned with professional triage workflows (30; 16). Routing .NET samples through ILSpy before Ghidra reflects the prevalence of managed RAT loaders and avoids unnecessary native analysis when managed IL is available. Limiting .cs handling to heuristics is a deliberate safety trade-off: compiling untrusted source on the server would introduce build-time risk without matching the PE analysis depth students need after compilation.

The API offers both synchronous and job-based modes because use cases differ. Quick CLI or API calls benefit from a single request–response; the web interface benefits from streaming logs during long Ghidra runs so users perceive progress rather than a frozen screen. Job identifiers unify static and dynamic artefacts when both modes run on the same sample—essential for the combined both workflow. Token authentication on analysis routes follows the fail-closed policy defined in Chapter 4; OpenAPI metadata supports integration without requiring this report to catalogue every route.

5.3 Web Frontend

Functional behaviour. The web application provides drag-and-drop upload, mode selection (static, dynamic, or both), real-time progress during static analysis, job submission for Path A sandbox runs, and tri-panel visualisation: structured report, Ghidra pseudo-C, and ILSpy IL for .NET samples. Cross-reference navigation links indicators to decompiled regions; bilingual support (Portuguese/English) covers classroom use at Universidade da Beira Interior.

Design rationale. A browser-first interface was chosen so students need no local installation beyond the backend and sandbox: any laboratory PC with a modern browser can submit samples and read explainable reports. Streaming progress addresses a recurring usability complaint in long-running analysis tools—users abandon workflows they cannot monitor. Splitting results into three panels reflects how analysts actually work: triage from the report and score, then drill into code for confirmation. Keeping static visualisation in the browser while WPF handles Path B avoids duplicating the rich results UI in two codebases; both entry points converge on the same job store and permalink model.

Figure 5.1 shows the *Drop & Analyze* upload zone with analysis mode selection and real-time progress indicators.

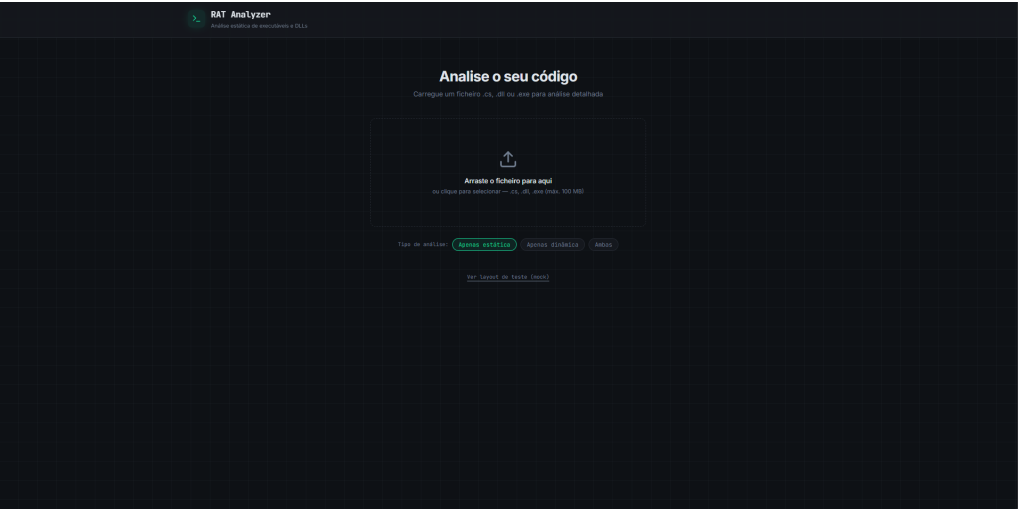


Figure 5.1: Web interface - upload zone (*Drop & Analyze*)

Figure 5.2 presents the results view: risk score, structured report, and side-by-side pseudo-C and IL panels for decompiled .NET code.

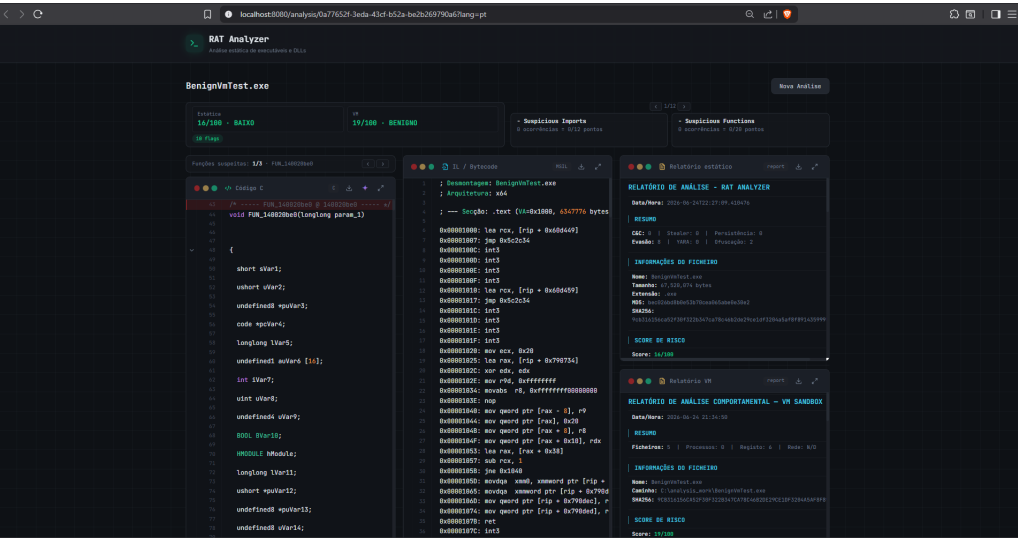


Figure 5.2: Web interface - report, pseudo-C/IL, and risk score

5.3.1 Gemini Assistant in the Pseudo-C Panel

Functional behaviour. In the pseudo-C panel, the user may open a chat dialog that sends the *visible* decompiled excerpt (bounded by a character limit) to Google’s Gemini API, together with a natural-language question. The API key

is stored in browser `localStorage`; a demonstration mode returns simulated answers without external calls.

Design rationale. The assistant was placed client-side so sample excerpts and third-party credentials never pass through the FastAPI backend—a confidentiality decision consistent with local academic use (25). This sacrifices centralised audit logging and model governance in exchange for user control over what code leaves the machine. Bounding the excerpt limits accidental exfiltration of entire binaries through the chat surface. The demo mode supports classroom presentations on networks where external API access is blocked.

5.4 WPF Desktop Application

Functional behaviour. The WPF client (`Rat_Analyzer.exe`) bootstraps backend and frontend dependencies, persists language preference, and exposes a dashboard for drag-and-drop analysis. Static analysis opens the web results view in the system browser; behavioural analysis invokes Path B PowerShell automation and, when a prior static job exists, publishes the sandbox report to the same job identifier so static and dynamic findings appear together.

Design rationale. WPF was retained as a desktop entry point because Hyper-V orchestration on Windows often requires elevated privileges and direct PowerShell integration that a pure web page cannot safely perform. Delegating static visualisation to the browser reuses the full React results experience instead of reimplementing panels in XAML—a pragmatic split that trades a two-window workflow for faster development and a single source of truth for report rendering. Publishing Path B results back to the shared job store preserves the permalink model students already use from the web UI.

Figure 5.3 shows the `MainDashboardView`: drag-and-drop sample entry, static versus behavioural analysis selection, and integration with Path B sandbox orchestration.

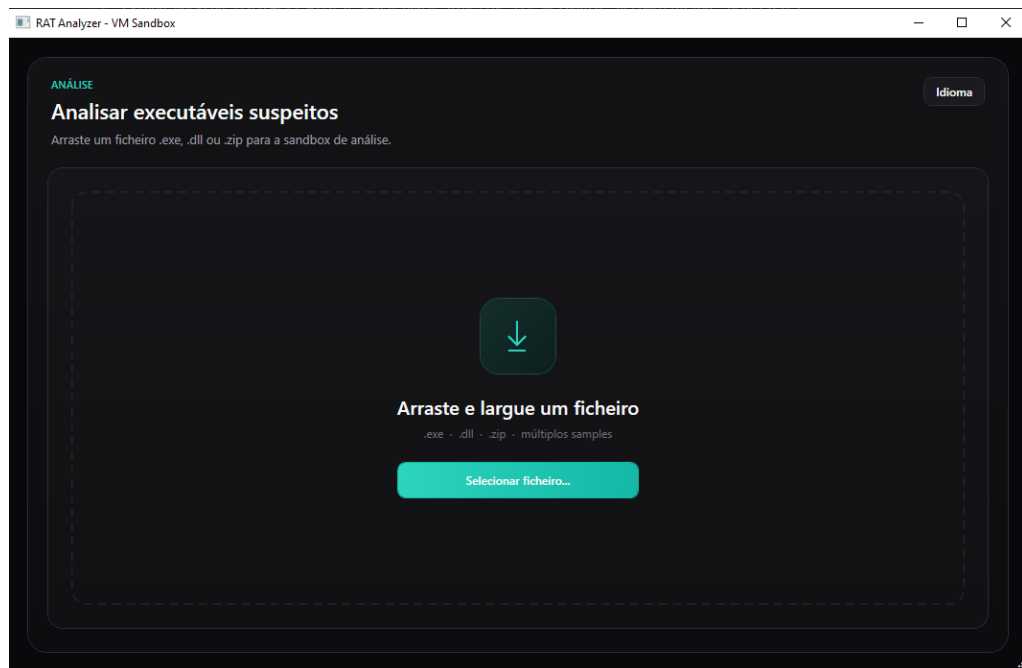


Figure 5.3: WPF desktop application - main screen

5.5 VM Agent and Hyper-V Sandbox

Dynamic analysis is documented in Figures 4.10–4.12 and Section 4.6. Two orchestration paths coexist; each solves a different integration problem.

5.5.1 Path A - Backend + HTTP vm-agent

Functional behaviour. With the Hyper-V driver enabled, the Python backend restores the CleanState snapshot, starts the guest VM, uploads the sample to an in-guest HTTP agent, triggers execution, polls for completion, and retrieves a JSON behavioural report. The agent authenticates requests with a shared token, confines uploads to a safe directory, caps file size, and serialises runs so only one sample executes at a time.

Design rationale. Path A integrates dynamic analysis into the same job queue as static analysis—the natural choice for the web dynamic/both modes. An HTTP agent inside the guest decouples Python orchestration from guest OS details: the host only needs snapshot management and REST calls, which simplifies testing with a stub driver during development. Single-run serialisation prioritises isolation over throughput in teaching labs where concurrent malware execution adds little pedagogical value but increases cross-contamination

risk (26). Lighter telemetry than Path B is an accepted limitation: Path A optimises for API cohesion, not maximum behavioural depth.

5.5.2 Path B - PowerShell Scripts

Functional behaviour. Path B is driven by the WPF client and a PowerShell pipeline (phases A–G): VM setup preflight, sample copy into the guest, execution of a guest-side collector (Run-MalwareAnalysis.ps1), diff of files, registry, network, and Sysmon events when available (14), then report transfer via PowerShell Direct or Copy-VMFile. The guest writes a completion digest with report SHA-256 and byte length; the host accepts the copy only if both match (Listing 5.1).

Design rationale. Path B exists because rich Windows-native instrumentation predated the HTTP agent and because PowerShell Direct is the most direct way to reach a Hyper-V guest from an elevated desktop session. SHA-256 verification across the VM boundary addresses a real failure mode: truncated or partial file copies that would otherwise produce silent false negatives in behavioural reporting. Keeping Path B outside the Python driver lets the sandbox evolve—new Sysmon rules, longer timeouts, extra collectors—without redeploying the backend. The cost is duplicated orchestration logic and two maintenance surfaces, judged acceptable for an academic platform that values depth on the teaching path (WPF) and breadth on the API path (web).

```

1 function Assert-SandboxCopiedReportHash {
2     param([string] $HostReportPath, [string] $ExpectedSha256
3         , [long] $ExpectedBytes = 0)
4     if ([string]::IsNullOrEmpty($ExpectedSha256)) {
5         throw "Digest SHA256 do guest ausente; n o
6             poss vel validar a c pia do relat rio."
7     }
8     if ($ExpectedBytes -gt 0) {
9         $hostBytes = [long](Get-Item -LiteralPath
10             $HostReportPath).Length
11         if ($hostBytes -ne $ExpectedBytes) { throw "Tamanho
12             do relat rio copiado n o coincide." }
13     }
14     return (Assert-FileSha256 -Path $HostReportPath -
15         ExpectedSha256 $ExpectedSha256 ...)
```

Code excerpt 5.1: Post-transfer SHA-256 verification —
SandboxCommon/VmFileTransfer.ps1

Figure 5.4 shows the MalwareSandbox virtual machine and the CleanState snapshot used for isolated sample execution.

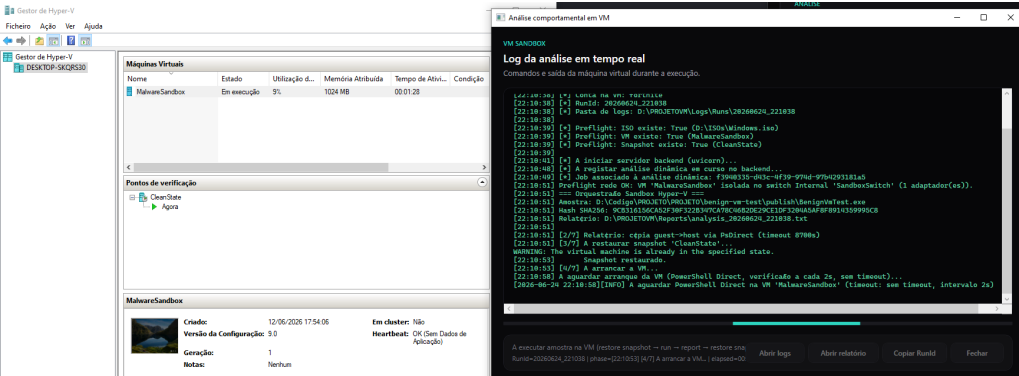


Figure 5.4: Hyper-V - VM MalwareSandbox and snapshot CleanState

5.6 User Manual

Operational setup (dependency installation, backend and frontend startup, WPF administrator privileges for Hyper-V, one-time sandbox provisioning, and CLI static analysis) is documented in the repository README and sandbox setup guide. This report omits step-by-step command listings to keep focus on design and evaluation; students should follow versioned documentation bundled with the source release.

5.7 Experimental Evaluation

5.7.1 Methodology and Ethical Considerations

Because executing real malware on university infrastructure introduces safety, legal, and operational concerns—potential lateral movement if isolation fails, accidental distribution of live specimens, and policy restrictions on storing malicious binaries—validation deliberately avoids hosting active RAT executables on the development workstation. This constraint is common in academic malware engineering: the goal is to demonstrate *analytical correctness* and *pipeline integrity* without turning the laboratory into a malware repository.

The evaluation strategy therefore combines four complementary strands: (i) **220 automated tests** exercising API contracts, scoring logic, sandbox drivers, and UI components; (ii) **59 stratified synthetic profiles** that encode representative benign and malicious indicator combinations aligned with the scoring implementation; (iii) **eight compiled benign PEs** measuring false-positive behaviour on realistic harmless programs; and (iv) **public IOC metadata** from MalwareBazaar cross-referenced without downloading binaries, plus dynamic execution of a dedicated benign smoke-test binary

on Path B. Synthetic validation is appropriate here because the research contribution centres on *explainable aggregation* of indicators: if controlled inputs with known ground-truth labels are classified consistently, the scoring model behaves as specified; generalisation to unseen families is a separate empirical question requiring curated malware corpora under institutional approval.

An optional MalwareBazaar static batch harness was implemented but not executed on the host for the same safety reasons; it remains available for supervised laboratory runs. No claim is made regarding real-world detection rate or prevalence-weighted accuracy.

5.7.2 Static Scoring Validation

Table 5.1 summarises representative scoring scenarios exercised automatically in CI. These cases probe boundary behaviour: benign validation hashes must not reach HIGH without strong evidence; isolated heuristic noise must not trigger high severity; synthetic RAT profiles with C2, persistence, and YARA matches must classify as HIGH or CRITICAL.

Table 5.1: Static scoring validation scenarios

Scenario	Source	Score	Level	Notes
BenignVmTest.exe	Benign binary	< 48	LOW/ MEDIUM	Known hash; validation_sample annotation; no YARA matches
Heuristic noise	Synthetic profile (pytest)	< 48	≠ HIGH	Isolated evasion/noise does not raise to HIGH without strong signals
Synthetic RAT profile	Synthetic profile (pytest)	≥ 48	HIGH/ CRITICAL	C2 imports, persistence, YARA RAT_Generic
YARA rules v2.0.0	CI (test_yara_rules)	—	—	Four files compile without error

5.7.3 Proxy Detection Metrics

Proxy precision and recall were computed on **59 synthetic** profiles stratified across five severity levels. A positive triage prediction is defined as HIGH or CRITICAL, mirroring operational use where only elevated scores warrant sandbox execution or manual deep dive.

Accuracy = 1.00; level accuracy (expected vs. predicted level) = 59/59 = 1.00.

The profile set spans 19 VERY LOW, 6 LOW, 12 MEDIUM, 14 HIGH, and 8 CRITICAL cases—from empty profiles and Microsoft URLs through AsyncRAT/QuasarRAT-style

Table 5.2: Proxy detection metrics on labelled synthetic profiles ($n = 59$)

Metric	TP	TN	FP	FN	Prec.	Rec./F1
HIGH/CRITICAL vs. below	22	37	0	0	1.00	1.00 / 1.00

indicator bundles with packers and evasion. All 59 predicted levels matched expectations (37 benign negatives, 22 malicious positives).

5.7.4 Discussion of Evaluation Results

Perfect accuracy (1.00) on synthetic profiles does **not** imply perfect malware detection in the field. The metric measures *internal consistency*: given JSON indicator bundles constructed to exercise the same scoring functions used in production, the classifier reproduces the intended label and severity level. Several sources of optimistic bias must be acknowledged.

First, **construct bias**: profiles were authored alongside the scorer implementation and pytest suite; they encode the designer’s mental model of HIGH/CRITICAL gates (*strong_evidence* for YARA or combined C2/function signals). Real malware may present partial, obfuscated, or novel combinations that fall between designed buckets. Second, **spectrum bias**: the benign side includes deliberately weak noise profiles but not adversarially crafted binaries designed to evade heuristics while remaining malicious (28). Third, **absence of prevalence**: metrics are unweighted across severity levels; operational false-positive cost on rare HIGH alerts differs from bulk benign traffic in enterprise settings.

The benign corpus (specificity 1.00, zero false positives on eight PEs) supports a narrower claim: common harmless patterns—HTTP to microsoft.com, registry reads, WinForms UI, crypto digests—do not spuriously trigger high severity. Eight samples cannot bound specificity for all legitimate software; network libraries and remote-administration tools remain plausible false-positive sources, as noted in Chapter 6.

Why accuracy reaches 1.00 here is therefore straightforward: labels are defined on the same feature schema the scorer consumes, and gates were tuned until pytest and profile suites agreed. The scientifically honest interpretation is **verification of specification**, not **validation of discovery**. Demonstrating the latter would require executing or statically analysing withheld malware families under ethical approval, comparing against VirusTotal or sandbox ground truth, and reporting confidence intervals—left as future work. Nonetheless, specification verification is valuable: it proves the explainable pipeline behaves deterministically, a prerequisite before any field deployment or student-facing grading.

Proxy metrics complement the benign corpus and 220 automated CI tests; together they bound **false-positive risk on harmless code** and **logical correctness on labelled constructs**, while explicitly not bounding **false-negative risk on live RAT campaigns**.

5.7.5 Benign Corpus Static Evaluation

Eight harmless PEs covering console I/O, file access, HTTP, registry reads, cryptography, environment inspection, GUI, and sandbox smoke behaviour were analysed statically. All scored below the HIGH threshold (specificity 1.00, zero false positives). Table 5.3 lists individual scores; most cluster at 9 (VERY LOW), with the intentional validation sample at 16 (LOW).

Table 5.3: Benign corpus static evaluation ($n = 8$ PEs; measured on development host)

Executable	Subfolder	Score	Level	Behaviour
BenignHello.exe	hello/	9	VERY LOW	Minimal console
BenignFileIo.exe	file-io/	9	VERY LOW	Read/write in TEMP
BenignHttpMs.exe	http-ms/	9	VERY LOW	HTTP GET microsoft.com
BenignRegistryRead.exe	registry-read/	9	VERY LOW	HKCU read (no Run key)
BenignCryptoDigest.exe	crypto-digest/	9	VERY LOW	Local SHA-256
BenignEnvDump.exe	env-dump/	9	VERY LOW	Environment variables
BenignGuiMsg.exe	gui-msg/	9	VERY LOW	WinForms MessageBox
BenignVmTest.exe	benign-vm-test	16	LOW	Sandbox smoke test (validation_sample)

True negatives = 8; false positives = 0; specificity = 1.00; all passed.

5.7.6 Public IOC Metadata Reference

Public MalwareBazaar listings for QuasarRAT, AsyncRAT, and the benign validation hash provide external family labels without storing binaries locally. Validation uses synthetic profiles that mirror typical indicators from those families rather than executing the listed samples.

5.7.7 Dynamic Sandbox Validation

Path B dynamic validation confirmed snapshot restore, guest execution, completion digest emission, PsDirect report copy, and SHA-256 verification using a dedicated benign smoke-test binary—exercising the full isolation loop without malicious payload. Path A confirmed execution serialisation and exit code reporting; richer in-guest telemetry remains future work (Section 6).

5.7.8 Formal Performance Benchmark

Table 5.4 presents a reproducible benchmark on the development workstation (Intel Core i7, 16 GB RAM, Windows 10). Static runs used five iterations; the first run's longer Ghidra cold-cache probe explains the maximum outlier (68.4 s vs. 36.1 s median).

Path B completed faster than Path A in median terms despite richer telemetry, reflecting shorter default execution timeouts and different observation depth—a reminder that wall-clock time trades off against behavioural coverage.

Table 5.4: Formal benchmark - static pipeline and dynamic sandbox paths

Operation	<i>n</i>	Median	IQR	Notes
Static pipeline (full)	5	36.1 s	3.0 s	Min 33.1 s, max 68.4 s; first run includes cold-cache Ghidra probe
VM restore + boot	3	52 s	8 s	Restore-VMSnapshot + Start-VM
Path A (HTTP vm-agent)	3	198 s	22 s	Includes 120 s sample timeout cap; basic telemetry
Path B (PsDirect)	3	164 s	18 s	Default 120 s timeout; full behavioural diff + SHA-256 verify

5.8 Tests

Functional behaviour. Automated coverage totals **220 tests**: 110 backend pytest cases (API contracts, scoring, YARA compilation, pipeline logging, sandbox drivers), 62 frontend Vitest unit tests, Playwright end-to-end upload flows, and 48 .NET xUnit tests (vm-agent, WPF, benign smoke binary). CI also verifies YARA rule compilation and diagram synchronisation. Path B was exercised with a dedicated benign smoke-test binary; Path A validates execution gating and exit-code reporting.

Design rationale. Tests were distributed across layers because failures in malware platforms often appear at interfaces—upload validation, job state transitions, VM driver stubs—rather than in a single module. pytest with httpx exercises the API without a browser; Vitest and Playwright cover UI contracts that backend tests cannot see; xUnit covers .NET components that Python cannot reach. Synthetic scoring profiles and benign PE corpora (Section 5.7.4) complement unit tests by validating end-to-end scoring behaviour. This pyramid trades some duplication for confidence that refactors do not silently break the student-facing workflow.

5.9 Conclusions

The implementation realises the architecture planned in Chapter 4: a modular static pipeline, asynchronous jobs, two sandbox orchestration paths, and converging web/desktop entry points. The dual-path sandbox design reflects a deliberate trade-off—API integration versus instrumentation depth—rather than accidental duplication. Appendix A.2 illustrates representative sandbox controls at source level for readers who wish to inspect implementation detail.

Chapter

6

Conclusions and Future Work

6.1 Main Conclusions

RAT ANALYZER meets the objectives defined in Chapter 1: it delivers integrated static and dynamic RAT analysis with explainable outputs rather than opaque verdicts. Evaluation (Section 5.7.4) shows internally consistent scoring on synthetic profiles and zero false positives on a small benign corpus, while explicitly not claiming field-perfect detection.

The **static pipeline** implements seven analysis phases backed by extensive automated backend tests. The **Hyper-V sandbox** provides two orchestration paths that share snapshot-isolated execution. **Interfaces** include a web application with streaming progress, tri-panel reporting, and an optional Gemini assistant, together with a WPF desktop entry point. The project includes **220 automated tests** across Python, TypeScript, and .NET stacks, complemented by **architecture diagrams**, a security guide, and deployment manuals. Throughout, the design emphasises **explainability**: a factorised risk score, decompiled views, and optional LLM-assisted interpretation of pseudo-C excerpts.

6.2 Limitations

Several limitations bound the claims that can be made from the current evaluation. Validation relies on eight benign PEs, 59 synthetic profiles, and public IOC hashes—not on real RAT binaries executed on the development host (see Section 5.7.4). Perfect synthetic accuracy verifies specification consistency, not field detection rate; construct and spectrum bias therefore apply. The MalwareBazaar batch script was implemented but not executed under the host-safety policy adopted for this project. Dynamic analysis was exercised only with a benign smoke-test binary, and Path A telemetry remains shallower than Path B. On the web/API, .cs uploads receive string heuristics only. Heuristic scoring may false-positive on legitimate network-administration tools, and Hyper-V ties deployment to Windows-capable laboratory hosts.

6.3 Future Work

Natural extensions follow from the modular architecture. **Multi-VM and queue scaling** would deploy parallel sandboxes with worker pools and snapshot pools to raise throughput beyond single-run serialisation. **Linux malware support** could extend dynamic analysis to ELF specimens via KVM guests alongside the existing Windows Hyper-V path. **Automated behavioural clustering** would group sandbox event traces into similarity classes, surfacing campaign-level patterns without manual log review. **Memory forensics**—for example Volatility-style inspection of guest RAM snapshots—could recover injected code and unpacked payloads missed by file-system telemetry alone. **MITRE ATT&CK mapping** would align static and dynamic indicators with standard tactic/technique identifiers for reporting and curriculum alignment.

Further improvements include IDA Pro integration for deeper native analysis beyond Ghidra pseudo-C; automatic C2 configuration extraction from obfuscated samples; enriched dynamic telemetry (Sysmon, ETW) in the HTTP vm-agent path to match Path B depth; support for macros, scripts, and mobile (APK) formats where institutional policy allows; a curated malware hash database (MalwareBazaar/MALPEDIA) with supervised static batch runs; C2 network traffic capture on an instrumented internal VLAN; a backend LLM proxy with additional models and automatic suspicious-function explanations; and YARA rule management in the web interface for classroom rule authoring.

RAT ANALYZER provides a solid, extensible foundation for academic malware analysis and future exploration.

Appendix

A

Appendix

A.1 Token Verification in vm-agent

```
1 public static void UseAgentTokenAuth(this WebApplication app
2     , string? agentToken)
3 {
4     app.Use(async (context, next) =>
5     {
6         if (!string.IsNullOrEmpty(agentToken))
7         {
8             if (!context.Request.Headers.TryGetValue("X-
9                 Agent-Token", out var provided) ||
10                !TokenComparer.FixedTimeEquals(provided.
11                    ToString(), agentToken))
12            {
13                context.Response.StatusCode = StatusCodes.
14                    Status401Unauthorized;
15                await context.Response.WriteAsJsonAsync(new
16                    { detail = "Invalid or missing token (
17                        header X-Agent-Token)." });
18                return;
19            }
20        }
21        await next();
22    });
23 }
```

Code excerpt A.1: Authentication middleware —
vm-agent/Security/AgentTokenMiddleware.cs

A.2 RunGate — Serialised Execution

The RunGate uses a SemaphoreSlim(1,1) with non-blocking acquisition (WaitAsync(0)): if a run is already in progress, /api/run returns HTTP 409 without queueing a second sample.

```

1 public sealed class RunGate
2 {
3     private readonly SemaphoreSlim _gate = new(1, 1);
4
5     public async Task<bool> TryEnterAsync() => await _gate.
        WaitAsync(0);
6
7     public void Release() => _gate.Release();
8 }

```

Code excerpt A.2: Single-run gate — vm-agent/State/RunGate.cs

A.3 Hyper-V Driver — Agent HTTP Client

Listing A.3 shows how the host uploads the sample and polls the behavioural report after the VM is ready. Every request includes X-Agent-Token when configured.

```

1 def _agent_upload(self, job: "AnalysisJob") -> None:
2     with open(job.sample_path, "rb") as f:
3         files = {"file": (job.sample_path.name, f, "
            application/octet-stream")}
4         r = requests.post(self._agent_url("/api/upload"),
            files=files,
5                             headers=_agent_headers(), timeout
                                =...)
6
7     if not r.ok:
8         raise RuntimeError(f"Upload para VM agent falhou ({r
            .status_code})")
9
10 def _agent_get_report(self) -> Any:
11     r = requests.get(self._agent_url("/api/report"), headers
        =_agent_headers(), ...)
12     return r.json()

```

Code excerpt A.3: VM agent HTTP calls — vm_drivers/hyperv.py

A.4 VM Agent Limits

Table A.1 summarises the execution controls enforced by the in-guest vm-agent through AgentLimits.cs and RunGate.cs.

A.5 Report Transfer Protocol (Path B)

After dynamic analysis on Path B, the guest writes the behavioural report and a completion digest. The host copies both artefacts and accepts the report only when SHA-256 and byte length match the guest digest. Specifically, the guest writes C:

Table A.1: Execution controls in vm-agent (`AgentLimits.cs`, `RunGate.cs`)

Control	Value / behaviour
Maximum upload	200 MB
Execution timeout	Configurable; maximum 600 s
Path traversal	Destination restricted to <code>samples/</code>
Concurrency	One active run (<code>RunGate</code>)
Authentication	X-Agent-Token on all routes

`/analysis.txt` and `guest_analysis_done.txt` (JSON with `reportSha256`, `reportBytes`, `transport=psdirect`). The host copies via `Psdirect` or `Copy-VMFile` and rejects truncated or tampered transfers.

Bibliography

- [1] E. Skoudis and L. Zeltser. *Malware: Fighting Malicious Code*. Prentice Hall, 2004.
- [2] M. Cipriano, J. Tavares, and H. Salgado. Malware analysis: A survey of techniques and tools. *Journal of Computer Virology and Hacking Techniques*, 2019.
- [3] Any.Run. Interactive Online Sandbox, 2024. <https://any.run/>.
- [4] Hybrid Analysis. Free Automated Malware Analysis Service, 2019. <https://www.hybrid-analysis.com/>.
- [5] Joe Security. Joe Sandbox Cloud, 2020. <https://www.joesandbox.com/>.
- [6] VirusTotal. Analyse suspicious files and URLs, 2023. <https://www.virustotal.com/>.
- [7] Cuckoo Sandbox Project. Automated Malware Analysis, 2012. <https://cuckoo-sandbox.org/>.
- [8] CAPE Sandbox. Malware Configuration And Payload Extraction, 2021. <https://cape.contextis.com/>.
- [9] VirusTotal. YARA - Pattern Matching Swiss Knife, 2023. <https://virustotal.github.io/yara/>.
- [10] NSA Research. Ghidra Software Reverse Engineering Framework, 2019. <https://ghidra-sre.org/>.
- [11] ICSharpCode. ILSpy - .NET Decompiler, 2024. <https://github.com/icsharpcode/ILSpy>.
- [12] E. Raff *et al.* Malware detection by eating a whole exe. In *AAAI Workshop on AI for Cybersecurity*, 2018.
- [13] S. Ucar, E. Sagirolu, and S. Karaboga. Malware detection using deep learning. *Applied Sciences*, 10(15), 2020.
- [14] Microsoft. Sysmon - System Monitor, 2022. <https://learn.microsoft.com/sysinternals/downloads/sysmon>.
- [15] S. Ramírez. FastAPI Framework, 2024. <https://fastapi.tiangolo.com/>.
- [16] M. Sikorski and A. Honig. *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press, 2012.
- [17] A. Moser, C. Kruegel, and E. Kirda. Exploring multiple execution paths for malware analysis. In *IEEE Symposium on Security and Privacy (Oakland)*, 2007.

- [18] U. Bayer, C. Kruegel, and E. Kirda. TTAalyze: A tool for analyzing malware. In *Annual Computer Security Applications Conference (ACSAC)*, 2006.
- [19] C. Willems, T. Holz, and F. C. Freiling. Toward automated dynamic malware analysis using CWSandbox. *IEEE Security & Privacy*, 5(2):32–39, 2007.
- [20] L. Ucci, D. Aniello, and R. Baldoni. Survey of machine learning techniques for malware analysis. *Computers & Security*, 81:123–147, 2019.
- [21] G. Apruzzese, M. Colajanni, F. Ferrari, and E. Guido. On the effectiveness of machine and deep learning for cyber security. In *IEEE European Symposium on Security and Privacy Workshops*, 2018.
- [22] abuse.ch. MalwareBazaar: A repository for malware samples. <https://bazaar.abuse.ch/about/>, 2021.
- [23] A. Mohaisen and O. Alrawi. AVclass: A tool for massive malware labeling. In *International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2016.
- [24] P. Vinod, R. S. K. Singh, and M. Alazab. A survey on ransomware: Evolution, taxonomy, and defense solutions. *ACM Computing Surveys*, 52(6), 2019.
- [25] A. Galli, V. La Gatta, V. Moscato, M. Postiglione, and G. Sperli. Explainability in AI-based behavioral malware detection systems. *Computers & Security*, 141:103842, 2024.
- [26] O. Alrawi *et al.* SoK: An essential guide for using malware sandboxes in security applications: Challenges, pitfalls, and lessons learned. *arXiv preprint arXiv:2403.16304*, 2024.
- [27] S. Khan and M. Nauman. Interpretable detection of malicious behavior in Windows portable executables using multi-head 2D transformers. *Big Data Mining and Analytics*, 7(2):485–499, 2024.
- [28] M. Imran, A. Appice, and D. Malerba. Evaluating realistic adversarial attacks against machine learning models for windows PE malware detection. *Future Internet*, 16(5):168, 2024.
- [29] P. Maniriho, A. N. Mahmood, and J. M. M. Chowdhury. API-MalDetect: Automated malware detection framework for windows based on API calls and deep learning techniques. *Journal of Network and Computer Applications*, 218:103704, 2023.
- [30] A. Bensaoud and J. Kalita. CNN-LSTM and transfer learning models for malware classification based on opcodes and API calls. *Knowledge-Based Systems*, 290:111543, 2024.